

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: CPU Scheduling

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Scheduling Concepts and Performance Criteria	2
1.1	CPU-I/O Burst Cycle	2
1.2	Scheduling Criteria	2
2	CPU Scheduling Algorithms	2
2.1	First-Come, First-Served (FCFS)	2
2.2	Shortest Job First (SJF)	2
2.3	Round Robin (RR) Scheduling	3
2.4	Priority Scheduling	3
2.5	Multilevel Queue (MLQ) & Multilevel Feedback Queue (MLFQ)	3
3	Multiprocessor Scheduling	3

1. Scheduling Concepts and Performance Criteria

In a single-processor system, only one process can run at a time; others must wait until the CPU is free. The objective of multiprogramming is to run multiple processes concurrently to maximize CPU utilization.

1.1 CPU-I/O Burst Cycle

Process execution consists of a cycle of **CPU execution** (**CPU burst**) and **I/O wait** (**I/O burst**). Processes alternate between these two states.

- **CPU-bound Process:** Spends most of its time doing computations (long CPU bursts).
- **I/O-bound Process:** Spends most of its time waiting for I/O (short CPU bursts).

1.2 Scheduling Criteria

Operating systems select processes from the ready queue based on five key performance criteria: 1. **CPU Utilization:** The percentage of time the CPU is busy executing tasks (Goal: Maximize, 40% to 90%). 2. **Throughput:** The number of processes completed per unit time (Goal: Maximize). 3. **Turnaround Time (TAT):** The total interval from process submission to its completion (Goal: Minimize). 4. **Waiting Time (WT):** The total time a process spends waiting in the ready queue (Goal: Minimize). 5. **Response Time (RT):** The time from submission to the first CPU response (Goal: Minimize, critical for interactive systems).

2. CPU Scheduling Algorithms

CPU scheduling decisions take place under four conditions: 1. When a process switches from running to waiting (Non-Preemptive). 2. When a process switches from running to ready (Preemptive). 3. When a process switches from waiting to ready (Preemptive). 4. When a process terminates (Non-Preemptive).

2.1 First-Come, First-Served (FCFS)

- **Principle:** The process that requests the CPU first gets it first. Managed via a FIFO queue.
- **Drawback:** Non-preemptive. Suffers from the **Convoy Effect** — short processes must wait for a single long process to finish, causing high average waiting times.

2.2 Shortest Job First (SJF)

- **Principle:** Associates with each process the length of its next CPU burst. The process with the shortest burst is scheduled.
- **SJF (Non-preemptive):** Once CPU is allocated, it cannot be preempted.
- **SRTF (Preemptive SJF):** If a new process arrives with a shorter remaining burst than the currently running process, the CPU is preempted.
- **Advantage:** SJF is **optimal** — it gives the minimum average waiting time for a given set of processes.

- **Disadvantage:** Cannot be implemented directly because the length of the next CPU burst is unknown. Prone to **starvation** of long processes.

2.3 Round Robin (RR) Scheduling

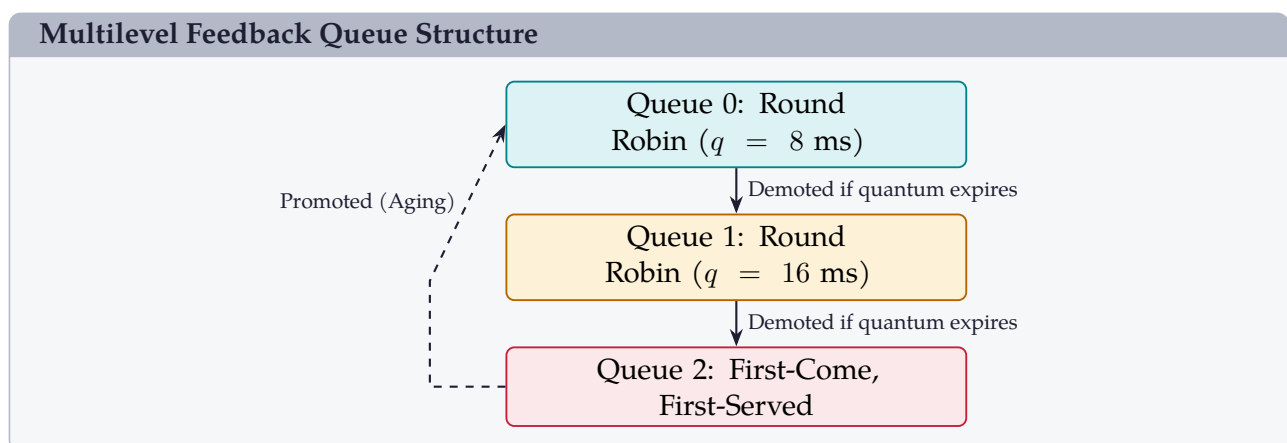
- **Principle:** Designed for time-sharing systems. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval up to **1 time quantum (slice)** (q).
- **Working:** If process burst $> q$, it is preempted and put back at the tail of the ready queue.
- **Performance Dependency:** If q is extremely large, RR behaves like FCFS. If q is extremely small, it causes high context-switching overhead.

2.4 Priority Scheduling

- **Principle:** A priority number is associated with each process, and the CPU is allocated to the process with the highest priority (lowest integer usually represents highest priority).
- **Problem: Starvation (Indefinite Blocking)** of low-priority processes.
- **Solution: Aging** — gradually increasing the priority of processes that wait in the system for a long time.

2.5 Multilevel Queue (MLQ) & Multilevel Feedback Queue (MLFQ)

- **MLQ:** Partitioned ready queue into several separate queues (e.g., foreground interactive and background batch) with fixed priority. No movement between queues.
- **MLFQ:** Allows processes to move between queues based on CPU burst history. If a process uses too much CPU time, it is demoted. If it waits too long, it is promoted (aging).



3. Multiprocessor Scheduling

When multiple CPUs are available, scheduling complexity increases:

- **Asymmetric Multiprocessing:** One master processor handles all scheduling decisions, I/O processing, and system activities; others run only user code. Simple, but master is a bottleneck.

- **Symmetric Multiprocessing (SMP):** Each processor schedules itself from the common ready queue or private ready queue.
- **Processor Affinity:** A process has an affinity for the processor it is currently running on to avoid the cache invalidation cost of migrating to another CPU.
 - **Soft Affinity:** OS tries to keep process on same CPU but does not guarantee it.
 - **Hard Affinity:** Process specifies it can only run on a subset of CPUs.
- **Load Balancing:** Keeps workload evenly distributed across all CPUs.
 - **Push Migration:** A specific task monitors load and pushes processes from overloaded CPUs to idle ones.
 - **Pull Migration:** An idle processor pulls a waiting task from a busy processor.

Quick Revision — Module 4

CPU Scheduling | OE-EC604B

Core Scheduling Formulas

Turnaround Time (TAT) = Completion Time (CT) – Arrival Time (AT)

Waiting Time (WT) = Turnaround Time (TAT) – Burst Time (BT)

Response Time (RT) = First CPU Start Time – Arrival Time (AT)

$$\text{Average Waiting Time} = \frac{1}{N} \sum_{i=1}^N WT_i$$

One-Line Definitions

- **Throughput:** Number of completed processes per unit time.
- **Convoy Effect:** Scenario in FCFS where short jobs wait behind a very long job, degrading latency.
- **Preemptive Scheduling:** OS mechanism to interrupt a running process and assign CPU to another.
- **Quantum (q):** The fixed time slice allocated to a process in Round Robin scheduling.
- **Starvation:** Situation where a process waits indefinitely because other processes have higher priorities.
- **Aging:** Technique of gradually increasing the priority of waiting processes to prevent starvation.
- **Processor Affinity:** Cache optimization keeping a process executing on the same CPU core.
- **Symmetric Multiprocessing (SMP):** Multiprocessing where each CPU makes its own scheduling decisions.

Frequently Asked / Viva Questions

Q1. Compare FCFS, SJF, and Round Robin scheduling algorithms.

FCFS is simple, non-preemptive, but prone to convoy effect. SJF is optimal for average waiting time but causes starvation and is hard to predict. Round Robin is preemptive, quantum-based, and ideal for interactive systems.

Q2. Show the Gantt chart and compute Turnaround and Waiting times for:

Frequently Asked / Viva Questions (contd.)

Process	Arrival Time	Burst Time	Priority
P1	0	8	3 (Low)
P2	1	4	1 (High)
P3	2	2	2

Using Preemptive Priority Scheduling:

Gantt Chart: P1 runs from [0-1]. At $t = 1$, P2 arrives with priority 1, preempts P1. P2 runs from [1-5]. At $t = 5$, P3 runs from [5-7]. At $t = 7$, P1 finishes from [7-14].

Completion Times (CT): P2 = 5, P3 = 7, P1 = 14.

Turnaround Times (TAT = CT - AT): P1 = 14, P2 = 4, P3 = 5. (Avg TAT = 7.67).

Waiting Times (WT = TAT - BT): P1 = 6, P2 = 0, P3 = 3. (Avg WT = 3.0).

Q3. What is the Convoy Effect in CPU scheduling? How is it resolved?

It happens in FCFS when one CPU-bound process holds the CPU while multiple I/O-bound processes wait in the ready queue. Resolved by using preemptive scheduling algorithms (like Round Robin).

Q4. Why is SJF scheduling called "optimal"? Why is it difficult to implement?

It is optimal because it mathematically minimizes the average waiting time for a set of processes. It is hard to implement because the exact burst time of future processes cannot be known in advance (it must be estimated using exponential smoothing).

Q5. How does the performance of Round Robin scheduling depend on the time quantum?

If the quantum is too large, it degenerates into FCFS. If it is too small, it causes excessive context-switching overhead, which wastes CPU cycles. A rule of thumb is that 80% of CPU bursts should be shorter than the quantum.

Q6. What is starvation? How does aging solve it?

Starvation occurs when low-priority processes never get scheduled because high-priority processes keep arriving. Aging solves this by gradually increasing the priority of a process the longer it remains waiting in the ready queue.

Q7. Differentiate between Multilevel Queue (MLQ) and Multilevel Feedback Queue (MLFQ).

In MLQ, processes are permanently assigned to a queue based on type, with no queue switching. In MLFQ, processes can move dynamically between queues based on CPU behavior, enabling automatic detection of I/O vs CPU bound jobs.

Q8. Explain Processor Affinity and its types.

Processor affinity is the policy of keeping a process on the same CPU to leverage cached data. Soft affinity means the OS tries to enforce this but can migrate the process. Hard affinity guarantees the process runs only on designated CPUs.

Frequently Asked / Viva Questions (contd.)

Q9. What is the difference between Push and Pull migration in load balancing?

Push migration runs a task that periodically checks load and "pushes" processes from busy CPUs to idle ones. Pull migration occurs when an idle CPU queries other CPUs and "pulls" an waiting process to execute.

Q10. Explain CPU-I/O burst cycle.

A process's lifecycle alternates between CPU execution blocks (CPU bursts) and waiting for device I/O actions (I/O bursts), ending with a final CPU burst and termination request.

CPU Scheduling Algorithms Comparison

Algorithm	Preemptive?	Starvation Risk?	Optimal average WT?
FCFS	No	No	No
SJF	No	Yes	Yes (among non-preemptive)
SRTF	Yes	Yes	Yes (optimal)
Round Robin	Yes	No	No
Priority	Yes/No	Yes	No